Development of a System for Detecting AI-Generated Images Using Deep Learning Methods

Afanasyev Kirill and Amangeldi Beksultan

Astana IT University, Big Data Analytics, BDA-2403
Astana, Kazakhstan

241064@astanait.edu.kz 241227@astanait.edu.kz

Abstract

Detectors for AI-generated images can lose accuracy when the generator or image processing changes. We compared a 50-layer residual network (ResNet-50) with a linear classifier on frozen Contrastive Language-Image Pre-training (CLIP) features. After duplicate removal, the TinyGenImage mirror contained 34,998 images. We held out Midjourney from training and validation. Both models used the same test images and three training seeds. In a follow-up, we continued ResNet-50 from epoch 1 to epoch 40. Mean F1 on original seen images rose from 0.910 to 0.982. On original unseen images, mean F1 fell from 0.310 to 0.166. CLIP reached 0.746 unseen F1. Its lead over epoch-40 ResNet-50 was 0.581, with a paired 95% confidence interval of [0.560, 0.601]. Unseen false-positive rates were 0.016 for ResNet-50 and 0.061 for CLIP. JPEG recompression reduced mean unseen F1 for both models. Downsampling to 112×112 raised unseen false-positive rates to 0.070 for ResNet-50 and 0.561 for CLIP. Both models fell short of the 0.80 unseen F1 target. The follow-up restarted Adam. Its script defaults to CUDA. The mirror includes one held-out generator and earlier pilot images.

Keywords: AI-generated images; deep learning; image forensics; generator generalization; processing robustness; false-positive rate

1 Introduction

The appearance of an image gives limited information about its origin. The National Institute of Standards and Technology describes detection, provenance tracking and content labelling as complementary methods [1]. In controlled experiments, Nightingale and Farid found that people struggled to distinguish generated faces from real faces [2]. Their tests focused on faces. Our tests cover AI-generated still images across multiple categories.

A false positive labels a real photograph as generated. A false negative leaves a generated image unflagged. A model can produce few false positives by labelling most images as real. We therefore report precision, recall, F1 and error counts with the false-positive rate (FPR). In the intended verification system, a positive decision would prompt a check of the image source and provenance.

FaceForensics++ tests facial manipulation at several compression levels and image sizes. Corvi et al. test compression and resizing for images from diffusion models [3, 4]. Frank et al. identify spectral patterns in generated images. Durall et al. link differences in spectra to upsampling [5, 6]. These results motivate processing tests for the detectors in this study.

Learning on Gradients and Neighboring Pixel Relationships use gradients or nearby pixels to detect artifacts [7, 8]. DIRE and AEROBLADE assess how a diffusion model or an autoencoder reconstructs an image [9, 10]. These methods use different training and test protocols.

FatFormer adapts CLIP to detect forgery evidence. Cozzolino et al. report transfer with a lightweight detector based on pretrained CLIP features [11, 12]. They also show why evaluation at a fixed threshold matters. A model can rank images well and still make many classification errors at a given threshold.

Our aim is to develop a system to detect AI-generated images. We compare a fine-tuned ResNet-50 with a linear classifier on frozen CLIP features. We evaluate generator transfer, JPEG compression, downsampling and false alarms on real photographs. The study addresses two questions.

1. **RQ1.** How well do ResNet-50 and the CLIP linear classifier detect AI-generated images on the seen and unseen test sets, based on F1 and false-positive rate?
2. **RQ2.** How do JPEG compression and downsampling affect each model's ability to detect AI-generated images, as measured by F1 and false-positive rate?

For RQ1, unseen F1 must be at least 0.80. FPR must be at most 0.10 on both original-image tests. The F1 drop from the seen test to the unseen test must be at most 0.10. For RQ2, we measure F1 and FPR on the same images before and after each operation.

We use six development generators and reserve Midjourney for the unseen test. We use paired intervals to assess uncertainty in differences between models and conditions. The 40-epoch follow-up compares the initial and final ResNet-50 checkpoints on the same images. Differences in training, source and resolution limit conclusions about architecture and generator identity.

Section 2 reviews the literature. Section 3 describes the experiment. Section 4 reports the results. Section 5 gives the conclusions. Sections 6 and 7 describe access to data and code.

2 Literature Review

Testing an unfamiliar generator, a new category from a familiar generator, and a compressed version of an existing input measures different capabilities. The studies below address these problems through surveys, transfer experiments and alternative detection features.

2.1 Reviews and evaluation scope

Foo et al. review generated media across several modalities [13]. Chen et al. cover diffusion-based generation and editing [14]. We limit our task to binary detection of ordinary still images. Video detection, source attribution and localization of partial edits need different labels and evaluation.

Zhang et al., Mahara and Rishe, and Xu et al. discuss spatial, frequency and multimodal evidence, including weaknesses on unfamiliar inputs [15–17]. Mahara and Rishe distinguish generator, category and scene shifts. Xu et al. separate transfer from robustness and explanation. We follow these distinctions by treating generator transfer and processing robustness as separate research questions.

Pei et al. compare facial benchmarks [18], and Heidari et al. review deep learning methods across media [19]. Facial and temporal clues need separate testing before they can be applied to object images. Shahzad et al. describe earlier detection methods [20]. Singh et al. discuss dataset bias and deployment costs [21]. Deng et al. examine detection alongside authentication, watermarking and disruption [22]. These reviews support reporting false positives and resource requirements together with detection scores.

2.2 Transfer beyond development data

Lamichhane tests pretrained vision transformers and examines the effects of pretraining and augmentation [23]. Saskoro et al. combine ResNet experts through a learned gate and evaluate additional generators [24]. Both studies motivate our use of pretrained representations and multiple development sources. Their internal scores describe performance on their own test data.

Coccomini et al. compare convolutional and CLIP-feature classifiers on diffusion images paired with different photographic sources. Their transfer results depend on the direction of the source change [25]. Bayram reports strong internal CIFAKE performance but much weaker results on external Tiny-GenImage data, where predictions concentrate on the generated class [26]. That error pattern makes false-positive rates especially relevant. In both studies, external testing can change more than generator identity.

Rout et al. evaluate attention and convolutional features on external forgery datasets that differ in domain and forgery type [27]. Sanders et al. hold out categories while keeping Stable Diffusion XL as the generator [28]. Sanders et al. isolate category changes. Our test holds out one generator and includes differences in photographic source and resolution.

2.3 Additional features and uncertainty

Man and Cho combine CLIP content features with wavelets and attention [29]. Yu and Xu fuse image and texture information [30]. SCADET uses frequency attention and contrastive spectral analysis for artwork [31]. Their component tests suggest that texture can add information to semantic features. The artwork results motivate a separate test of texture features on ordinary photographs.

Yumlembam et al. combine uncertainty measures with optimized rejection [32]. Rejecting uncertain inputs can reduce errors and also reduce the number of images that receive a decision. Tests of rejection should report both errors and coverage. We use a fixed threshold and give a decision for every test image. Both configurations use the same test IDs and three seeds. We test compression and downsampling separately.

3 Materials and Methods

3.1 Data, allocation and provenance

We use all 35,000 rows of the third-party Tiny-GenImage mirror [33, 34]. We pin revision `89c4fe9efd0e`. On 6 November 2026, we checked the downloaded parquet shards against their published SHA256 digests. We read the image bytes from the shards and pooled the source training and validation partitions. We formed the new partitions with sampling seed 20261106.

We screened exact files, decoded RGB pixels and pixels after common encoding. This removed two duplicate photographs. Table 1 lists the final counts. Each partition uses separate IDs and exact hashes. The dataset may contain perceptual duplicates. The official GenImage archives still require a separate audit.

We reserve all 2,500 Midjourney images for the unseen test. ADM, BigGAN, GLIDE, SD15, VQDM and Wukong supply 2,500 generated images each. These six generators supply development data and 500 images each for the seen test. The mirror contains no SD14 rows. Within a test, both models, all seeds and all processing conditions use the same IDs. The seen and unseen tests use different images.

The initial plan specified 20,000 images. The full-mirror experiment changes the split, source access, development generators and fitted models. Earlier pilot images can occur in the new partitions. Git history retains the earlier 1,920-image analysis separately.

We convert each image to RGB and re-encode it at native size as JPEG quality 75 with 4:2:0 chroma subsampling. We call this the original condition. BigGAN images are 128×128. ADM, GLIDE and VQDM images are 256×256. SD15 and Wukong images are 512×512. Midjourney dimensions range from 512 to 1,024 pixels. Real photographs range from 34 to 4,786 pixels in width and 38 to 4,419 in height. Prior compression, image size and category can provide shortcuts for a detector [35].

Table 1 Study partitions before and after exact-file, decoded-RGB and common-encoding duplicate screening.

Partition	Nominal	Retained	Real	Generated
Training	20,000	19,998	9,998	10,000
Validation	4,000	4,000	2,000	2,000
Seen test	6,000	6,000	3,000	3,000
Unseen test	5,000	5,000	2,500	2,500
Total	35,000	34,998	17,498	17,500

Two duplicate photographs were removed before splitting. Midjourney occurs only in the unseen test; six other generators supply development and seen-test images. The cohort includes previously inspected pilot images.

3.2 Models and training

ResNet-50 starts from torchvision ImageNet1K V1 weights. We replace the final layer with one binary logit and update all parameters [36, 37]. CLIP ViT-B/32 uses OpenAI weights. We freeze its encoder and train a linear layer on the 512-dimensional features [38, 39].

We use bicubic interpolation to resize inputs to 224×224 pixels. We normalize the inputs for each model. We use Adam and binary cross-entropy with logits. The positive-class weight is the training real-to-generated ratio, $9,998/10,000 = 0.9998$. Both configurations use seeds 42, 43 and 44. Training uses the same manifest and no augmentation.

In the baseline, ResNet-50 trains for one epoch with batch size 16 and learning rate 10^{-4}. The CLIP head trains for up to 40 epochs with batch size 32 and learning rate 10^{-3}. We select the CLIP checkpoint with the highest validation F1 at threshold 0.50. A tie goes to the earlier epoch. The selected CLIP epochs are 38, 40 and 38.

The follow-up starts from the saved epoch-1 ResNet-50 weights. We initialize a new Adam optimizer at epoch 2 because the saved weights omit optimizer state. We continue the three runs to epoch 40 with the same batch size and learning rate. We record validation F1 for every epoch and evaluate the final checkpoint for each seed.

The baseline used an AMD EPYC 9V74 CPU. The encoders used bfloat16 autocast. Parameters, Adam state, the CLIP head and saved scores used float32. The continuation script defaults to CUDA with bfloat16 autocast. The submitted histories record loss and validation F1. They contain no device or precision metadata. The two model configurations differ in pretraining and trainable parameters.

3.3 Metrics and paired uncertainty

Generated images form the positive class. We calculate the metrics from true positives (TP), true negatives (TN), false positives (FP) and false negatives (FN).

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \qquad \text{Precision} = \frac{TP}{TP + FP}, \tag{1}$$

$$\text{Recall} = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2TP}{2TP + FP + FN}, \tag{2}$$

$$\text{FPR} = \frac{FP}{FP + TN}. \tag{3}$$

We set undefined precision and F1 to zero. We also report area under the receiver operating characteristic curve (AUROC) and average precision (AP). We apply threshold 0.50 and calculate each metric for each seed. The tables report means and sample standard deviations (SDs) across the three seeds.

We use 1,000 paired bootstrap resamples to calculate percentile 95% confidence intervals. We resample images within each label and source group. Each resample uses the same image IDs for both models and all three seeds. The intervals describe image-sampling uncertainty for the fitted checkpoints. The sample SD describes the observed differences between the three training seeds.

We use 9,999 two-sided paired permutations for the model comparison. For each image, we exchange model outputs jointly across seeds. If k absolute differences are at least as large as the observed difference, $p = (1 + k)/10,000$. The smallest reportable p-value is 0.0001. We treat comparisons beyond the original unseen test as exploratory and report their unadjusted p-values.

The practical comparison rule has three requirements. The mean F1 difference in absolute value must be at least 0.03. The confidence interval must exclude zero in the same direction. The model with higher F1 must have a mean FPR of at most 0.10 in that condition. The absolute performance targets apply separately.

3.4 Processing and decision threshold

We test the common quality-75 baseline and two processed versions. The first version uses JPEG quality-65 recompression. The second uses downsampling to 112×112 pixels. We apply each operation separately. Each version then receives the 224×224 input resize. We calculate each paired change as the processed metric minus the original metric.

The 40-epoch comparison uses the fixed threshold of 0.50. A separate baseline analysis compares this threshold with one selected on the original validation images. That archived analysis uses the original ResNet-50 and CLIP checkpoints.

4 Results

We evaluate 11,000 distinct test images. Each model has three seeds and three conditions. These evaluations yield 198,000 scores, including 99,000 scores for the new ResNet-50 checkpoints.

4.1 RQ1. Original images

At epoch 40, ResNet-50 achieves a mean seen F1 of 0.982 and an unseen F1 of 0.166 (Table 2). CLIP achieves 0.949 and 0.746, respectively. The seen-to-unseen F1 drops are 0.817 for ResNet-50 and 0.203 for CLIP. Both exceed the 0.10 limit. Every seed of both models meets the 0.10 FPR limit on both original-image tests. Each model falls short of the 0.80 unseen F1 target.

Mean unseen recall is 0.093 for ResNet-50 and 0.632 for CLIP. Mean unseen FPR is 0.016 and 0.061, respectively. ResNet-50 makes fewer false alarms and misses more generated images. Unseen AUROC reaches 0.787 for ResNet-50 and 0.925 for CLIP.

Table 2 Original-image metrics at threshold 0.5. Values are means ± sample SD over seeds 42, 43 and 44. P is precision and R is recall.

Model	Test	Accuracy	P	R	F1
ResNet-50 (epoch 40)	Seen	0.982 ± 0.003	0.981 ± 0.011	0.984 ± 0.013	0.982 ± 0.003
CLIP linear	Seen	0.949 ± 0.001	0.942 ± 0.001	0.957 ± 0.002	0.949 ± 0.001
ResNet-50 (epoch 40)	Unseen	0.539 ± 0.020	0.859 ± 0.015	0.093 ± 0.051	0.166 ± 0.080
CLIP linear	Unseen	0.785 ± 0.002	0.911 ± 0.002	0.632 ± 0.007	0.746 ± 0.004

Model	Test	FPR	AUROC	F1 95% CI	FPR 95% CI
ResNet-50 (epoch 40)	Seen	0.019 ± 0.011	0.998 ± 0.001	[0.980, 0.985]	[0.015, 0.023]
CLIP linear	Seen	0.059 ± 0.001	0.988 ± 0.000	[0.944, 0.955]	[0.050, 0.068]
ResNet-50 (epoch 40)	Unseen	0.016 ± 0.011	0.787 ± 0.018	[0.153, 0.180]	[0.012, 0.020]
CLIP linear	Unseen	0.061 ± 0.002	0.925 ± 0.000	[0.732, 0.759]	[0.052, 0.071]

Seen test: 3,000 images per class. Unseen test: 2,500 per class. We resample images within each label and source group. The intervals apply to these checkpoints.

4.2 Paired model differences

On the original unseen test, the mean CLIP-minus-ResNet F1 difference is 0.581. The paired 95% interval is [0.560, 0.601] and the permutation p-value is 0.0001 (Table 3). The difference exceeds 0.03, the interval is above zero and CLIP's FPR is 0.061. This meets the practical comparison rule. CLIP also meets the rule on JPEG-compressed unseen images. After downsampling, CLIP retains higher F1 and exceeds the 0.10 FPR limit.

ResNet-50 has higher F1 on all three seen conditions. Its largest advantage is 0.189 on the downsampled seen images. The comparisons apply to the evaluated checkpoints. The configurations differ in architecture, pretraining and trainable parameters.

Table 3 Paired F1 differences (CLIP minus ResNet-50), with image-bootstrap 95% intervals. Original unseen images are the main comparison. We report unadjusted p-values for the other comparisons.

Test	Condition	ΔF1	95% CI	p
Seen	Original	-0.033	[-0.039, -0.027]	0.0001
Seen	JPEG 65	-0.060	[-0.067, -0.052]	0.0001
Seen	Resize 112	-0.189	[-0.196, -0.181]	0.0001
Unseen	Original	0.581	[0.560, 0.601]	0.0001
Unseen	JPEG 65	0.547	[0.528, 0.568]	0.0001
Unseen	Resize 112	0.514	[0.500, 0.528]	0.0001

The Monte Carlo permutation resolution is 0.0001 (9,999 swaps plus the observed sample). An F1 advantage also needs an acceptable false-positive rate to meet the practical comparison rule.

4.3 RQ2. JPEG recompression and downsampling

On the unseen test, JPEG recompression reduces mean F1 from 0.166 to 0.140 for ResNet-50 and from 0.746 to 0.687 for CLIP. Mean FPR also falls, to 0.012 and 0.028, respectively. The 95% intervals for all four changes exclude zero (Table 4).

Downsampling raises unseen FPR from 0.016 to 0.070 for ResNet-50 and from 0.061 to 0.561 for CLIP (Figure 1). The increase for CLIP is 0.500, with a paired 95% interval of [0.480, 0.519]. After downsampling, CLIP flags more than half of the real photographs. ResNet-50 mean F1 rises to 0.219 on the unseen test and remains below the F1 target. CLIP mean F1 changes by −0.013, with a 95% interval of [−0.027, 0.001].

On the seen test, downsampling reduces ResNet-50 F1 to 0.945 and CLIP F1 to 0.756. It raises FPR to 0.066 for ResNet-50 and 0.564 for CLIP.

Table 4 Paired processing minus original changes, averaged over seeds, with 95% image-bootstrap intervals. Compression and downsampling are applied independently to the common quality-75 baseline.

JPEG recompression, quality 65

Model	Test	ΔF_1 [95% CI]	ΔFPR [95% CI]
ResNet-50 (epoch 40)	Seen	-0.002 $[-0.004, -0.001]$	-0.0039 $[-0.0060, -0.0020]$
CLIP linear	Seen	-0.029 $[-0.035, -0.023]$	-0.0292 $[-0.0364, -0.0226]$
ResNet-50 (epoch 40)	Unseen	-0.026 $[-0.035, -0.017]$	-0.0044 $[-0.0064, -0.0024]$
CLIP linear	Unseen	-0.059 $[-0.070, -0.047]$	-0.0337 $[-0.0413, -0.0268]$

Downsampling to 112×112 pixels

Model	Test	ΔF_1 [95% CI]	ΔFPR [95% CI]
ResNet-50 (epoch 40)	Seen	-0.037 $[-0.041, -0.034]$	$+0.0473$ $[0.0420, 0.0522]$
CLIP linear	Seen	-0.193 $[-0.200, -0.185]$	$+0.5049$ $[0.4864, 0.5223]$
ResNet-50 (epoch 40)	Unseen	$+0.053$ $[0.041, 0.065]$	$+0.0537$ $[0.0483, 0.0592]$
CLIP linear	Unseen	-0.013 $[-0.027, 0.001]$	$+0.4996$ $[0.4797, 0.5185]$

Each comparison uses identical IDs within the 6,000-image seen or 5,000-image unseen test. ΔFPR is shown to four decimals: the ResNet JPEG estimates differ before rounding. Read the changes in F1 and FPR together.

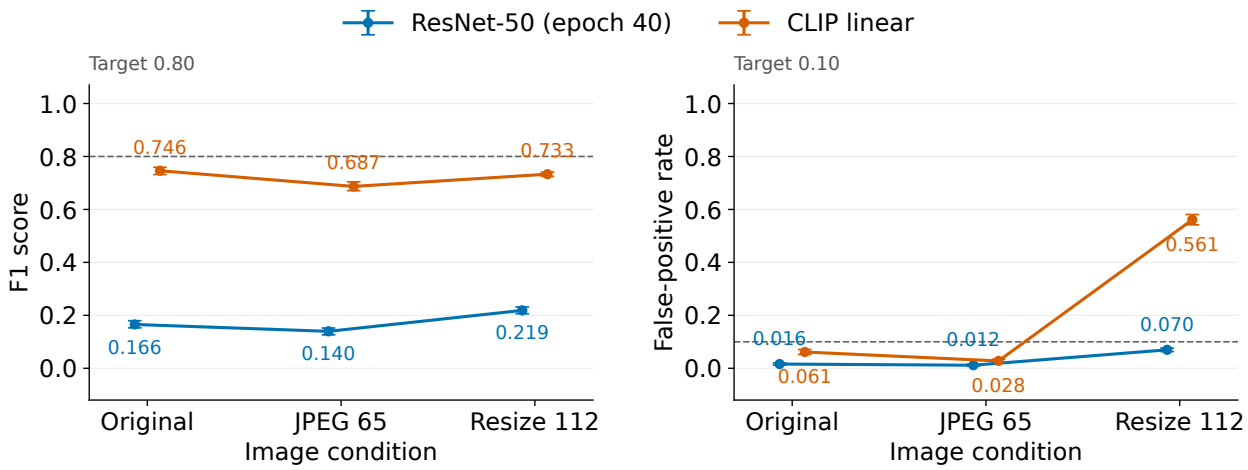


Figure 1 Mean unseen-test F1 and false-positive rate across three seeds. Error bars show 95% intervals from bootstrap resampling of images. Dashed lines mark the targets.

4.4 ResNet-50 at epochs 1 and 40

On original seen images, mean ResNet-50 F1 rises from 0.910 to 0.982. On original unseen images, it falls from 0.310 to 0.166 (Table 5). Mean unseen FPR falls from 0.096 to 0.016. Mean unseen recall falls from 0.206 to 0.093.

The unseen F1 change differs by seed. Seed 42 rises from 0.191 to 0.258. Seeds 43 and 44 fall from 0.392 and 0.348 to 0.127 and 0.112. On the development validation split, epoch-40 F1 is 0.980, 0.985 and 0.981. The validation improvement accompanies weaker average transfer to Midjourney. The Adam reset and possible backend differences limit attribution to training duration.

Table 5 ResNet-50 results at epochs 1 and 40. Values are means across the same three seeds at threshold 0.50.

Test	Condition	F1 (epoch 1)	F1 (epoch 40)	FPR (epoch 1)	FPR (epoch 40)
Seen	Original	0.910	0.982	0.098	0.019
Seen	JPEG 65	0.899	0.980	0.064	0.015
Seen	Resize 112	0.674	0.945	0.964	0.066
Unseen	Original	0.310	0.166	0.096	0.016
Unseen	JPEG 65	0.289	0.140	0.062	0.012
Unseen	Resize 112	0.666	0.219	0.971	0.070

The follow-up starts from the epoch-1 weights and initializes a new Adam optimizer. Both evaluations use the same test images.

4.5 Observed errors and reproducibility

Figure 2 shows seed 42 on original unseen images. ResNet-50 detects 380 of 2,500 generated images and falsely flags 71 of 2,500 real photographs. CLIP detects 1,593 generated images and falsely flags 158 real photographs.

We matched the new score files to the archived test set by image ID and label. The confusion counts, F1 and FPR agree with the submitted summary for all 18 ResNet groups. The public aggregate exports reproduce the five paper tables and both figures. The code can also recompute the statistics when it has the original per-image exports.

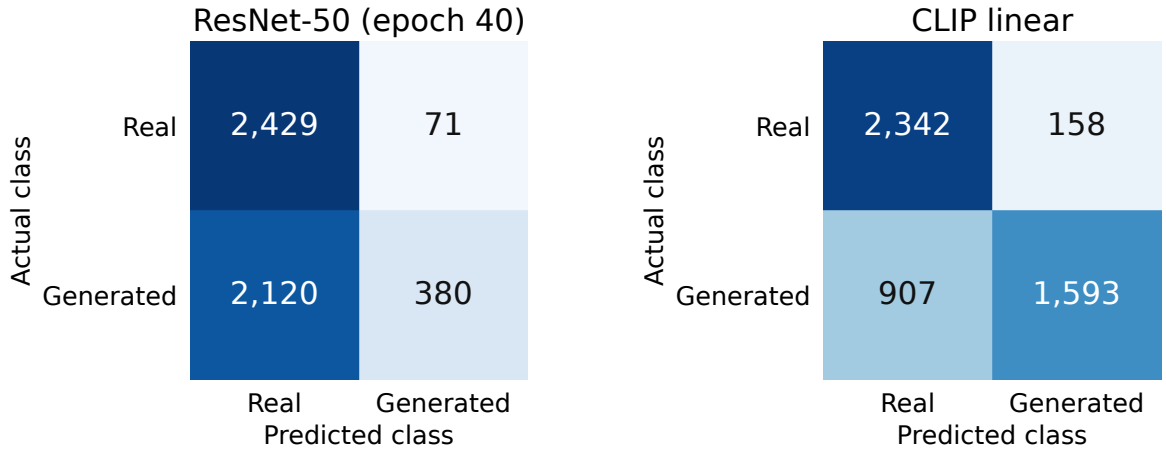


Figure 2 Confusion matrices for the original unseen test at threshold 0.50, seed 42. Rows are actual classes. Columns are predicted classes.

5 Conclusion

CLIP achieves higher F1 on the unseen generator. At epoch 40, ResNet-50 reaches higher F1 on the seen test and makes fewer false alarms on original unseen images. The epoch-40 ResNet-50 detects fewer Midjourney images on average than the epoch-1 baseline. Both configurations fall short of the unseen F1 and transfer targets.

JPEG recompression lowers unseen F1 for both configurations. Downsampling has the largest effect on false alarms for CLIP. Reporting F1 with FPR shows how many genuine photographs a detector would flag for an extra check. The rates are important for both original and processed inputs.

The study uses one mirror and one held-out generator. The splits contain differences in category, image size and compression history. Some source images appeared in earlier pilot analyses. Perceptual duplicates and possible exposure during pretraining still require an audit. The confidence intervals cover image sampling for these checkpoints. Three training seeds give a limited measure of variation across fitted runs.

Future tests can use fresh external data, several held-out generators and matched source characteristics. They can keep the backend and optimizer state constant when testing training duration. A validation experiment with a false-positive limit would help assess alternative operating thresholds. Image verification also requires source and provenance checks.

6 Data Availability

Source images are available in `TheKernel01/Tiny-GenImage`, revision 89c4fe9efd0ebc7ce5c7641ef5 7d578ccd639c69. The mirror declares Creative Commons Attribution-NonCommercial-ShareAlike 4.0. We accessed it on 6 November 2026.

The `C0nflu3nc3/RMT` repository contains the manifest, source-shard records, duplicate audit and original prediction archive in `experiments/full35k/`. The manifest SHA256 begins `1d8fb1ab8e174d31`. The new training histories, summary metrics, evaluation tables and figures are in `experiments/resnet40/`. The new per-image ResNet scores are separate from the public aggregate exports.

7 Code Availability

The `C0nflu3nc3/RMT` repository contains the analysis and training code. Install the packages in requirements-analysis.txt. To rebuild the tables and figures, run:

`python experiments/resnet40/reproduce.py`
Use --score-dir with the original epoch-40 exports to recompute the statistics. To verify the CPU baseline, run pythonreproduce_tables.py. The continuation script uses the pinned image cache and the saved epoch-1 weights. The code uses the MIT licence. Dataset and pretrained-model licences are separate.

References

[1] B. Chandra, J. Dunietz, K. Roberts, Y. Lee, P. Fontana and G. Awad. Reducing risks posed by synthetic content: An overview of technical approaches to digital content transparency. NIST AI 100-4, 2024. doi:10.6028/NIST.AI.100-4.

[2] S. J. Nightingale and H. Farid. AI-synthesized faces are indistinguishable from real faces and more trustworthy. PNAS, 119(8):e2120481119, 2022. doi:10.1073/pnas.2120481119.

[3] A. Rössler et al. FaceForensics++: Learning to detect manipulated facial images. ICCV, pp. 1–11, 2019. doi:10.1109/ICCV.2019.00009.

[4] R. Corvi et al. On the detection of synthetic images generated by diffusion models. ICASSP, pp. 1–5, 2023. doi:10.1109/ICASSP49357.2023.10095167.

[5] J. Frank et al. Leveraging frequency analysis for deep fake image recognition. ICML, 119:3247–3258, 2020.

[6] R. Durall, M. Keuper and J. Keuper. Watch your up-convolution: CNN based generative deep neural networks are failing to reproduce spectral distributions. CVPR, pp. 7887–7896, 2020. doi:10.1109/CVPR42600.2020.00791.

[7] C. Tan et al. Learning on gradients: Generalized artifacts representation for GAN-generated images detection. CVPR, pp. 12105–12114, 2023. doi:10.1109/CVPR52729.2023.01165.

[8] C. Tan et al. Rethinking the up-sampling operations in CNN-based generative network for generalizable deepfake detection. CVPR, pp. 28130–28139, 2024. doi:10.1109/CVPR52733.2024.02657.

[9] Z. Wang et al. DIRE for diffusion-generated image detection. ICCV, pp. 22388–22398, 2023. doi:10.1109/ICCV51070.2023.02051.

[10] J. Ricker, D. Lukovnikov and A. Fischer. AEROBLADE: Training-free detection of latent diffusion images using autoencoder reconstruction error. CVPR, pp. 9130–9140, 2024. doi:10.1109/CVPR52733.2024.00872.

[11] H. Liu et al. Forgery-aware adaptive transformer for generalizable synthetic image detection. CVPR, pp. 10770–10780, 2024. doi:10.1109/CVPR52733.2024.01024.

[12] D. Cozzolino et al. Raising the bar of AI-generated image detection with CLIP. CVPR Workshops, pp. 4356–4366, 2024. doi:10.1109/CVPRW63382.2024.00439.

[13] L. G. Foo, H. Rahmani and J. Liu. AI-generated content (AIGC) for various data modalities: A survey. ACM Computing Surveys, 57(9):243, 2025. doi:10.1145/3728633.

[14] H. Chen et al. Comprehensive exploration of diffusion models in image generation: A survey. Artificial Intelligence Review, 58(4):99, 2025. doi:10.1007/s10462-025-11110-3.

[15] Y. Zhang et al. Unmasking AI-created visual content: A review of generated images and deepfake detection technologies. Journal of King Saud University Computer and Information Sciences, 37(6):148, 2025. doi:10.1007/s44443-025-00154-8.

[16] A. Mahara and N. Rishe. Methods and trends in detecting AI-generated images: A comprehensive review. Computer Science Review, 60:100908, 2026. doi:10.1016/j.cosrev.2026.100908.

[17] Q. Xu et al. Advancements in AI-generated content forensics: A systematic literature review. ACM Computing Surveys, 58(3):60, 2025. doi:10.1145/3760526.

[18] G. Pei et al. Deepfake generation and detection: A benchmark and survey. ACM Computing Surveys, 58(11):273, 2026. doi:10.1145/3801962.

[19] A. Heidari et al. Deepfake detection using deep learning methods: A systematic and comprehensive review. WIREs Data Mining and Knowledge Discovery, 14(2):e1520, 2024. doi:10.1002/widm.1520.

[20] H. F. Shahzad et al. A review of image processing techniques for deepfakes. Sensors, 22(12):4556, 2022. doi:10.3390/s22124556.

[21] L. H. Singh, P. Charanarur and N. K. Chaudhary. Advancements in detecting deepfakes: AI algorithms and future prospects - a review. Discover Internet of Things, 5(1):53, 2025. doi:10.1007/s43926-025-00154-0.

[22] J. Deng et al. A survey of defenses against AI-generated visual media: Detection, disruption, and authentication. ACM Computing Surveys, 58(5):123, 2026. doi:10.1145/3770916.

[23] D. Lamichhane. Advanced detection of AI-generated images through vision transformers. IEEE Access, 13:3644–3652, 2025. doi:10.1109/ACCESS.2024.3522759.

[24] R. A. F. Saskoro, N. Yudistira and T. N. Fatyanosa. Detection of AI-generated images from various generators using gated expert convolutional neural network. IEEE Access, 12:147772–147783, 2024. doi:10.1109/ACCESS.2024.3466614.

[25] D. A. Coccomini et al. Detecting images generated by diffusers. PeerJ Computer Science, 10:e2127, 2024. doi:10.7717/peerj-cs.2127.

[26] F. Bayram. Color space analysis for AI-generated image detection on CIFAKE: CNN models and statistical evaluation. IEEE Access, 14:30747–30757, 2026. doi:10.1109/ACCESS.2026.3665315.

[27] J. Rout et al. Stacked multi-fusion CNN: An adaptive attention model for privacy preserving deepfake forensics. Scientific Reports, 16:24653, 2026. doi:10.1038/s41598-026-55325-y.

[28] B. Sanders, D. Morrison and D. Harris-Birtill. Exploring techniques to distinguish between real images and those generated using Stable Diffusion XL. PLOS ONE, 21(1):e0339917, 2026. doi:10.1371/journal.pone.0339917.

[29] Q. Man and Y.-I. Cho. Transformer based on multi-domain feature fusion for AI-generated image detection. Electronics, 15(3):716, 2026. doi:10.3390/electronics15030716.

[30] H. Yu and B. Xu. Multi-modal texture fusion network for detecting AI-generated images. Frontiers in Artificial Intelligence, 8:1663292, 2025. doi:10.3389/frai.2025.1663292.

[31] X. Zhang, Z. Yu and J. Zhao. SCADET: A detection framework for AI-generated artwork integrating dynamic frequency attention and contrastive spectral analysis. PLOS ONE, 20(11):e0336328, 2025. doi:10.1371/journal.pone.0336328.

[32] R. Yumlembam et al. Detection of AI generated images using combined uncertainty measures and particle swarm optimised rejection mechanism. Scientific Reports, 15:44021, 2025. doi:10.1038/s41598-025-28572-8.

[33] M. Zhu et al. GenImage: A million-scale benchmark for detecting AI-generated image. NeurIPS, 36:77771–77782, 2023. doi:10.52202/075280-3398.

[34] TheKernel01. Tiny-GenImage dataset. Revision 89c4fe9efd0e; accessed 6 November 2026. Mirror-declared licence CC BY-NC-SA 4.0. https://huggingface.co/datasets/TheKernel01/Tiny-GenImage.

[35] P. Grommelt et al. Fake or JPEG? Revealing common biases in generated image detection datasets. ECCV 2024 Workshops, LNCS 15644, pp. 80–95, 2025. doi:10.1007/978-3-031-92089-9_6.

[36] K. He et al. Deep residual learning for image recognition. CVPR, pp. 770–778, 2016. doi:10.1109/CVPR.2016.90.

[37] S.-Y. Wang et al. CNN-generated images are surprisingly easy to spot... for now. CVPR, pp. 8692–8701, 2020. doi:10.1109/CVPR42600.2020.00872.

[38] A. Radford et al. Learning transferable visual models from natural language supervision. ICML, 139:8748–8763, 2021.

[39] U. Ojha, Y. Li and Y. J. Lee. Towards universal fake image detectors that generalize across generative models. CVPR, pp. 24480–24489, 2023. doi:10.1109/CVPR52729.2023.02345.

AI assistance declaration. We used OpenAI ChatGPT to prepare code, present numerical results, and format the article in LaTeX. It also helped draft and edit the article and search the literature. The reported results come from archived model predictions and evaluation exports.